

```
=====

object.__uediv__(self, other)
object.__ifloordiv__(self, other)
object.__imod__(self, other)
object.__ipow__(self, other[, modulo])
object.__lshift__(self, other)
object.__rshift__(self, other)
object.__iand__(self, other)
object.__ixor__(self, other)
object.__ior__(self, other)
```

These methods are called to implement the augmented arithmetic assignments ("+=", "-=", "*=", "@=", "/=", "//=", "%=", "**=", "<=", ">=", "&=", "^=", "|="). These methods should attempt to do the operation in-place (modifying *self*) and return the result (which could be, but does not have to be, *self*). If a specific method is not defined, or if that method returns "NotImplemented", the augmented assignment falls back to the normal methods. For instance, if *x* is an instance of a class with an "__iadd__()" method, "x += y" is equivalent to "x = x.__iadd__(y)". If "__iadd__()" does not exist, or if "x.__iadd__(y)" returns "NotImplemented", "x.__add__(y)" and "y.__radd__(x)" are considered, as with the evaluation of "x + y". In certain situations, augmented assignment can result in unexpected errors (see Why does a_tuple[i] += [-item-] raise an exception when the addition works?), but this behavior is in fact part of the data model.

```
object.__neg__(self)
object.__pos__(self)
object.__abs__(self)
object.__invert__(self)
```

Called to implement the unary arithmetic operations ("-", "+", "abs()" and "~").

```
object.__complex__(self)
object.__int__(self)
object.__float__(self)
```

Called to implement the built-in functions "complex()", "int()" and "float()". Should return a value of the appropriate type.

```
object.__index__(self)
```

Called to implement "operator.index()", and whenever Python needs to losslessly convert the numeric object to an integer object (such as in slicing, or in the built-in "bin()", "hex()" and "oct()" functions). Presence of this method indicates that the numeric object is an integer type. Must return an integer.

If "__int__()", "__float__()" and "__complex__()" are not defined then corresponding built-in functions "int()", "float()" and "complex()" fall back to "__index__()".

```
object.__round__(self[, ndigits])
object.__trunc__(self)
object.__floor__(self)
object.__ceil__(self)
```

Called to implement the built-in function "round()" and "math" functions "trunc()", "floor()" and "ceil()". Unless *ndigits* is passed to "__round__()" all these methods should return the value of the object truncated to an "Integral" (typically an "int").

Changed in version 3.14: "int()" no longer delegates to the "__trunc__()" method.

With Statement Context Managers

=====

A *context manager* is an object that defines the runtime context to be established when executing a "with" statement. The context manager handles the entry into, and the exit from, the desired runtime context for the execution of the block of code. Context managers are normally invoked using the "with" statement (described in section The with statement), but can also be used by directly invoking their methods.

Typical uses of context managers include saving and restoring various kinds of global state, locking and unlocking resources, closing opened files, etc.

For more information on context managers, see Context Manager Types. The "object" class itself does not provide the context manager methods.

object.__enter__(self)

Enter the runtime context related to this object. The "with" statement will bind this method's return value to the target(s) specified in the "as" clause of the statement, if any.

object.__exit__(self, exc_type, exc_value, traceback)

Exit the runtime context related to this object. The parameters describe the exception that caused the context to be exited. If the context was exited without an exception, all three arguments will be "None".

If an exception is supplied, and the method wishes to suppress the exception (i.e., prevent it from being propagated), it should return a true value. Otherwise, the exception will be processed normally upon exit from this method.

Note that "__exit__()" methods should not reraise the passed-in exception; this is the caller's responsibility.

See also:

NOTE (pdf2): paragraph added at revision, line 109.

****PEP 343**** - The "with" statement

The specification, background, and examples for the Python "with" statement.

Customizing positional arguments in class pattern matching

=====

When using a class name in a pattern, positional arguments in the pattern are not allowed by default, i.e. "case MyClass(x, y)" is typically invalid without special support in "MyClass". To be able to

use that kind of pattern, the class needs to define a `* __match_args__ *` attribute.

object.`__match_args__`

This class variable can be assigned a tuple of strings. When this class is used in a class pattern with positional arguments, each positional argument will be converted into a keyword argument, using the corresponding value in `* __match_args__ *` as the keyword. The absence of this attribute is equivalent to setting it to `()`.

For example, if `"MyClass.__match_args__"` is `("left", "center", "right")` that means that `"case MyClass(x, y)"` is equivalent to `"case MyClass(left=x, center=y)"`. Note that the number of arguments in the pattern must be smaller than or equal to the number of elements in `* __match_args__ *`; if it is larger, the pattern match attempt will raise a `"TypeError"`.

Added in version 3.10.

See also:

****PEP 634**** - Structural Pattern Matching

The specification for the Python `"match"` statement.

Emulating buffer types

=====

The buffer protocol provides a way for Python objects to expose efficient access to a low-level memory array. This protocol is implemented by builtin types such as `"bytes"` and `"memoryview"`, and third-party libraries may define additional buffer types.

While buffer types are usually implemented in C, it is also possible to implement the protocol in Python.

object.`__buffer__(self, flags)`

Called when a buffer is requested from `*self*` (for example, by the `"memoryview"` constructor). The `*flags*` argument is an integer representing the kind of buffer requested, affecting for example whether the returned buffer is read-only or writable. `"inspect.BufferFlags"` provides a convenient way to interpret the flags. The method must return a `"memoryview"` object.

****Thread safety:**** In `*free-threaded*` Python, implementations must manage any internal export counter using atomic operations. The method must be safe to call concurrently from multiple threads, and the returned buffer's underlying data must remain valid until the corresponding `"__release_buffer__()"` call completes. See Thread safety for `memoryview` objects for details.

object.`__release_buffer__(self, buffer)`

Called when a buffer is no longer needed. The `*buffer*` argument is a `"memoryview"` object that was previously returned by `"__buffer__()"`. The method must release any resources associated with the buffer. This method should return `"None"`.

****Thread safety:**** In `*free-threaded*` Python, any export counter decrement must use atomic operations. Resource cleanup must be

thread-safe, as the final release may race with concurrent releases from other threads.

Buffer objects that do not need to perform any cleanup are not required to implement this method.

Added in version 3.12.

See also:

****PEP 688**** - Making the buffer protocol accessible in Python
Introduces the Python "`__buffer__`" and "`__release_buffer__`" methods.

"collections.abc.Buffer"
ABC for buffer types.

Annotations =====

Functions, classes, and modules may contain *annotations*, which are a way to associate information (usually *type hints*) with a symbol.

object.`__annotations__`

This attribute contains the annotations for an object. It is lazily evaluated, so accessing the attribute may execute arbitrary code and raise exceptions. If evaluation is successful, the attribute is set to a dictionary mapping from variable names to annotations.

Changed in version 3.14: Annotations are now lazily evaluated.

object.`__annotate__`(format)

An *annotate function*. Returns a new dictionary object mapping attribute/parameter names to their annotation values.

Takes a format parameter specifying the format in which annotations values should be provided. It must be a member of the "`annotationlib.Format`" enum, or an integer with a value corresponding to a member of the enum.

If an `annotate` function doesn't support the requested format, it must raise "`NotImplementedError`". `Annotate` functions must always support "`VALUE`" format; they must not raise "`NotImplementedError`()" when called with this format.

When called with "`VALUE`" format, an `annotate` function may raise "`NameError`"; it must not raise "`NameError`" when called requesting any other format.

If an object does not have any annotations, "`__annotate__`" should preferably be set to "`None`" (it can't be deleted), rather than set to a function that returns an empty dict.

Added in version 3.14.

See also:

****PEP 649**** · Deferred evaluation of annotation using descriptors
Introduces lazy evaluation of annotations and the "`__annotate__`"

function.

Special method lookup

=====

For custom classes, implicit invocations of special methods are only guaranteed to work correctly if defined on an object's type, not in the object's instance dictionary. That behaviour is the reason why the following code raises an exception:

```
... pass
...
>>> c = C()
>>> c.__len__ = lambda: 5
>>> len(c)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'C' has no len()
```

The rationale behind this behaviour lies with a number of special methods such as "`__hash__()`" and "`__repr__()`" that are implemented by all objects, including type objects. If the implicit lookup of these methods used the conventional lookup process, they would fail when invoked on the type object itself:

```
>>> 1.__hash__() == hash(1)
True
>>> int.__hash__() == hash(int)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: descriptor '__hash__' of 'int' object needs an argument
```

Incorrectly attempting to invoke an unbound method of a class in this way is sometimes referred to as `metaclass confusion`, and is avoided by bypassing the instance when looking up special methods:

```
>>> type(1).__hash__(1) == hash(1)
True
>>> type(int).__hash__(int) == hash(int)
True
```

In addition to bypassing any instance attributes in the interest of correctness, implicit special method lookup generally also bypasses the "`__getattr__()`" method even of the object's metaclass:

```
>>> class Meta(type):
...     def __getattr__(*args):
...         print("Metaclass getattr invoked")
...         return type.__getattr__(*args)
...
>>> class C(object, metaclass=Meta):
...     def __len__(self):
...         return 10
...     def __getattr__(*args):
...         print("Class getattr invoked")
...         return object.__getattr__(*args)
...
>>> c = C()
>>> c.__len__()          # Explicit lookup via instance
Class getattr invoked
10
```

```
>>> type(c).__len__(c)      # Explicit lookup via type
MetaClass getattr invoked
10
>>> len(c)                  # Implicit lookup
10
```

Bypassing the "`__getattr__()`" machinery in this fashion provides significant scope for speed optimisations within the interpreter, at the cost of some flexibility in the handling of special methods (the special method `*must*` be set on the class object itself in order to be consistently invoked by the interpreter).

STRING-METHODS

=====

String Methods

Strings implement all of the common sequence operations, along with the additional methods described below.

Strings also support two styles of string formatting, one providing a large degree of flexibility and customization (see "`str.format()`", Format string syntax and Custom string formatting) and the other based on C "`printf`" style formatting that handles a narrower range of types and is slightly harder to use correctly, but is often faster for the cases it can handle (printf-style String Formatting).

The Text Processing Services section of the standard library covers a number of other modules that provide various text related utilities (including regular expression support in the "`re`" module).

`str.capitalize()`

Return a copy of the string with its first character capitalized and the rest lowercased.

Changed in version 3.8: The first character is now put into titlecase rather than uppercase. This means that characters like digraphs will only have their first letter capitalized, instead of the full character.

`str.casefold()`

Return a casefolded copy of the string. Casefolded strings may be used for caseless matching.

Casefolding is similar to lowercasing but more aggressive because it is intended to remove all case distinctions in a string. For example, the German lowercase letter "`ß`" is equivalent to "`ss`". Since it is already lowercase, "`lower()`" would do nothing to "`ß`"; "`casefold()`" converts it to "`ss`". For example:

```
>>> 'straße'.lower()
'straße'
>>> 'straße'.casefold()
'strasse'
```

The casefolding algorithm is described in section 3.13 ·Default Case Folding· of the Unicode Standard.

Added in version 3.3.

```
str.center(width, fillchar=' ', /)
```

Return centered in a string of length **width**. Padding is done using the specified **fillchar** (default is an ASCII space). The original string is returned if **width** is less than or equal to *"len(s)"*. For example:

```
>>> 'Python'.center(10)
' Python '
>>> 'Python'.center(10, '-')
'--Python--'
>>> 'Python'.center(4)
'Python'
```

```
str.count(sub[, start[, end]])
```

Return the number of non-overlapping occurrences of substring **sub** in the range [**start**, **end**]. Optional arguments **start** and **end** are interpreted as in slice notation.

If **sub** is empty, returns the number of empty strings between characters which is the length of the string plus one. For example:

```
>>> 'spam, spam, spam'.count('spam')
3
>>> 'spam, spam, spam'.count('spam', 5)
2
>>> 'spam, spam, spam'.count('spam', 5, 10)
1
>>> 'spam, spam, spam'.count('eggs')
0
>>> 'spam, spam, spam'.count("")
17
```

```
str.encode(encoding='utf-8', errors='strict')
```

Return the string encoded to "bytes".

encoding defaults to *"utf-8"*; see Standard Encodings for possible values.

errors controls how encoding errors are handled. If *"strict"* (the default), a *"UnicodeError"* exception is raised. Other possible values are *"ignore"*, *"replace"*, *"xmlcharrefreplace"*, *"backslashreplace"* and any other name registered via *"codecs.register_error()"*. See Error Handlers for details.

For performance reasons, the value of **errors** is not checked for validity unless an encoding error actually occurs, Python Development Mode is enabled or a debug build is used. For example:

```
>>> encoded_str_to_bytes = 'Python'.encode()
>>> type(encoded_str_to_bytes)
<class 'bytes'>
>>> encoded_str_to_bytes
b'Python'
```

Changed in version 3.1: Added support for keyword arguments.

Changed in version 3.9: The value of the **errors** argument is now

checked in Python Development Mode and in debug mode.

`str.endswith(suffix[, start[, end]])`

Return "True" if the string ends with the specified **suffix**, otherwise return "False". **suffix** can also be a tuple of suffixes to look for. With optional **start**, test beginning at that position. With optional **end**, stop comparing at that position. Using **start** and **end** is equivalent to `"str[start:end].endswith(suffix)"`. For example:

```
>>> 'Python'.endswith('on')
True
>>> 'a tuple of suffixes'.endswith(('at', 'in'))
False
>>> 'a tuple of suffixes'.endswith(('at', 'es'))
True
>>> 'Python is amazing'.endswith('is', 0, 9)
True
```

See also `"startswith()"` and `"removesuffix()"`.

`str.expandtabs(tabsize=8)`

Return a copy of the string where all tab characters are replaced by one or more spaces, depending on the current column and the given tab size. Tab positions occur every **tabsize** characters (default is 8, giving tab positions at columns 0, 8, 16 and so on). To expand the string, the current column is set to zero and the string is examined character by character. If the character is a tab ("`\t`"), one or more space characters are inserted in the result until the current column is equal to the next tab position. (The tab character itself is not copied.) If the character is a newline ("`\n`") or return ("`\r`"), it is copied and the current column is reset to zero. Any other character is copied unchanged and the current column is incremented by one regardless of how the character is represented when printed. For example:

```
>>> '01\t012\t0123\t01234'.expandtabs()
'01  012  0123  01234'
>>> '01\t012\t0123\t01234'.expandtabs(4)
'01 012 0123 01234'
>>> print('01\t012\n0123\t01234'.expandtabs(4))
01 012
0123 01234
```

`str.find(sub[, start[, end]])`

Return the lowest index in the string where substring **sub** is found within the slice `"s[start:end]"`. Optional arguments **start** and **end** are interpreted as in slice notation. Return `"-1"` if **sub** is not found. For example:

```
>>> 'spam, spam, spam'.find('sp')
0
>>> 'spam, spam, spam'.find('sp', 5)
6
```

See also `"rfind()"` and `"index()"`.

Note:

The "find()" method should be used only if you need to know the position of *sub*. To check if *sub* is a substring or not, use the "in" operator:

```
>>> 'Py' in 'Python'
True
```

str.format(*args, **kwargs)

Perform a string formatting operation. The string on which this method is called can contain literal text or replacement fields delimited by braces "{}". Each replacement field contains either the numeric index of a positional argument, or the name of a keyword argument. Returns a copy of the string where each replacement field is replaced with the string value of the corresponding argument. For example:

```
>>> "The sum of 1 + 2 is {}".format(1+2)
'The sum of 1 + 2 is 3'
>>> "The sum of {a} + {b} is {answer}".format(answer=1+2, a=1, b=2)
'The sum of 1 + 2 is 3'
>>> "{1} expects the {0} Inquisition!".format("Spanish", "Nobody")
'Nobody expects the Spanish Inquisition!'
```

See Format string syntax for a description of the various formatting options that can be specified in format strings.

Note:

When formatting a number ("int", "float", "complex", "decimal.Decimal" and subclasses) with the "n" type (ex: "{:n}".format(1234)), the function temporarily sets the "LC_CTYPE" locale to the "LC_NUMERIC" locale to decode "decimal_point" and "thousands_sep" fields of "localeconv()" if they are non-ASCII or longer than 1 byte, and the "LC_NUMERIC" locale is different than the "LC_CTYPE" locale. This temporary change affects other threads.

Changed in version 3.7: When formatting a number with the "n" type, the function sets temporarily the "LC_CTYPE" locale to the "LC_NUMERIC" locale in some cases.

str.format_map(mapping, /)

Similar to "str.format(**mapping)", except that "mapping" is used directly and not copied to a "dict". This is useful if for example "mapping" is a dict subclass:

```
>>> class Default(dict):
...     def __missing__(self, key):
...         return key
...
>>> '{name} was born in {country}'.format_map(Default(name='Guido')) [rev pdf2]
'Guido was born in country'
```

Added in version 3.2.

str.index(sub[, start[, end]])

Like "find()", but raise "ValueError" when the substring is not found. For example:

```
>>> 'spam, spam, spam'.index('spam')
0
>>> 'spam, spam, spam'.index('eggs')
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    'spam, spam, spam'.index('eggs')
    ~~~~~~^~~~~~
ValueError: substring not found
```

See also "rindex()".

str.isalnum()

Return "True" if all characters in the string are alphanumeric and there is at least one character, "False" otherwise. A character "c" is alphanumeric if one of the following returns "True": "c.isalpha()", "c.isdecimal()", "c.isdigit()", or "c.isnumeric()". For example:

```
>>> 'abc123'.isalnum()
True
>>> 'abc123!@#'.isalnum()
False
>>> ''.isalnum()
False
>>> ' '.isalnum()
False
```

str.isalpha()

Return "True" if all characters in the string are alphabetic and there is at least one character, "False" otherwise. Alphabetic characters are those characters defined in the Unicode character database as `Letter`, i.e., those with general category property being one of `Lm`, `Lt`, `Lu`, `Li`, or `Lo`. Note that this is different from the Alphabetic property defined in the section 4.10 `Letters, Alphabetic, and Ideographic` of the Unicode Standard. For example:

```
>>> 'Letters and spaces'.isalpha()
False
>>> 'LettersOnly'.isalpha()
True
>>> 'μ'.isalpha() # non-ASCII characters can be considered alphabetical too
True
```

See Unicode Properties.

str.isascii()

Return "True" if the string is empty or all characters in the string are ASCII, "False" otherwise. ASCII characters have code points in the range U+0000-U+007F. For example:

```
>>> 'ASCII characters'.isascii()
True
>>> 'μ'.isascii()
False
```

Added in version 3.7.

str.isdecimal()

Return "True" if all characters in the string are decimal characters and there is at least one character, "False" otherwise. Decimal characters are those that can be used to form numbers in base 10, such as U+0660, ARABIC-INDIC DIGIT ZERO. Formally a decimal character is a character in the Unicode General Category `·Nd·`. For example:

```
>>> '0123456789'.isdecimal()
True
>>> '٠١٢٣٤٥٦٧٨٩'.isdecimal() # Arabic-Indic digits zero to nine
True
>>> 'alphabetic'.isdecimal()
False
```

`str.isdigit()`

Return "True" if all characters in the string are digits and there is at least one character, "False" otherwise. Digits include decimal characters and digits that need special handling, such as the compatibility superscript digits. This covers digits which cannot be used to form numbers in base 10, like the Kharosthi numbers. Formally, a digit is a character that has the property value `Numeric_Type=Digit` or `Numeric_Type=Decimal`.

For example:

```
>>> '0123456789'.isdigit()
True
>>> '٠١٢٣٤٥٦٧٨٩'.isdigit() # Arabic-Indic digits zero to nine
True
>>> '١'.isdigit() # Vulgar fraction one fifth
False
>>> '2'.isdecimal(), '٢'.isdigit(), '٢'.isnumeric()
(False, True, True)
```

See also `"isdecimal()"` and `"isnumeric()"`.

`str.isidentifier()`

Return "True" if the string is a valid identifier according to the language definition, section Names (identifiers and keywords).

`"keyword.iskeyword()"` can be used to test whether string "s" is a reserved identifier, such as `"def"` and `"class"`.

Example:

```
>>> from keyword import iskeyword

>>> 'hello'.isidentifier(), iskeyword('hello')
(True, False)
>>> 'def'.isidentifier(), iskeyword('def')
(True, True)
```

`str.islower()`

Return "True" if all cased characters [4] in the string are lowercase and there is at least one cased character, "False" otherwise.

`str.isnumeric()`

Return "True" if all characters in the string are numeric characters, and there is at least one character, "False" otherwise. Numeric characters include digit characters, and all characters that have the Unicode numeric value property, e.g. U+2155, VULGAR FRACTION ONE FIFTH. Formally, numeric characters are those with the property value Numeric_Type=Digit, Numeric_Type=Decimal or Numeric_Type=Numeric. For example:

```
>>> '0123456789'.isnumeric()
True
>>> '٠١٢٣٤٥٦٧٨٩'.isnumeric() # Arabic-Indic digits zero to nine
True
>>> '١'.isnumeric() # Vulgar fraction one fifth
True
>>> '2'.isdecimal(), '2'.isdigit(), '2'.isnumeric()
(False, True, True)
```

See also "isdecimal()" and "isdigit()".

str.isprintable()

Return "True" if all characters in the string are printable, "False" if it contains at least one non-printable character.

Here `.printable` means the character is suitable for `"repr()"` to use in its output; `.non-printable` means that `"repr()"` on built-in types will hex-escape the character. It has no bearing on the handling of strings written to `"sys.stdout"` or `"sys.stderr"`.

The printable characters are those which in the Unicode character database (see `"unicodedata"`) have a general category in group Letter, Mark, Number, Punctuation, or Symbol (L, M, N, P, or S); plus the ASCII space `0x20`. Nonprintable characters are those in group Separator or Other (Z or C), except the ASCII space.

For example:

```
>>> ".isprintable(), ' '.isprintable()
(True, True)
>>> '\t'.isprintable(), '\n'.isprintable()
(False, False)
```

See also `"isspace()"`.

str.isspace()

Return "True" if there are only whitespace characters in the string and there is at least one character, "False" otherwise.

For example:

```
>>> ".isspace()
False
>>> ' '.isspace()
True
>>> '\t\n'.isspace() # TAB and BREAK LINE
True
>>> '\u3000'.isspace() # IDEOGRAPHIC SPACE
True
```

A character is **whitespace** if in the Unicode character database

(see "unicodedata"), either its general category is "Zs" (·Separator, space·), or its bidirectional class is one of "WS", "B", or "S".

See also "isprintable()".

`str.istitle()`

Return "True" if the string is a titlecased string and there is at least one character, for example uppercase characters may only follow uncased characters and lowercase characters only cased ones. Return "False" otherwise.

For example:

```
>>> 'Spam, Spam, Spam'.istitle()
True
>>> 'spam, spam, spam'.istitle()
False
>>> 'SPAM, SPAM, SPAM'.istitle()
False
```

See also "title()".

`str.isupper()`

Return "True" if all cased characters [4] in the string are uppercase and there is at least one cased character, "False" otherwise.

```
>>> 'BANANA'.isupper()
True
>>> 'banana'.isupper()
False
>>> 'baNana'.isupper()
False
>>> ''.isupper()
False
```

`str.join(iterable, /)`

Return a string which is the concatenation of the strings in *iterable*. A "TypeError" will be raised if there are any non-string values in *iterable*, including "bytes" objects. The separator between elements is the string providing this method. For example:

```
>>> ', '.join(['spam', 'spam', 'spam'])
'spam, spam, spam'
>>> '-'.join('Python')
'P-y-t-h-o-n'
```

See also "split()".

`str.ljust(width, fillchar=' ', /)`

Return the string left justified in a string of length *width*. Padding is done using the specified *fillchar* (default is an ASCII space). The original string is returned if *width* is less than or equal to "len(s)".

For example:

```
>>> 'Python'.ljust(10)
'Python  '
>>> 'Python'.ljust(10, '.')
'Python....'
>>> 'Monty Python'.ljust(10, '.')
'Monty Python'
```

See also "rjust()".

`str.lower()`

Return a copy of the string with all the cased characters [4] converted to lowercase. For example:

```
>>> 'Lower Method Example'.lower()
'lower method example'
```

The lowercasing algorithm used is described in section 3.13 "Default Case Folding" of the Unicode Standard.

`str.lstrip(chars=None, /)`

Return a copy of the string with leading characters removed. The **chars** argument is a string specifying the set of characters to be removed. If omitted or "None", the **chars** argument defaults to removing whitespace. The **chars** argument is not a prefix; rather, all combinations of its values are stripped:

```
>>> '  spacious  '.lstrip()
'spacious  '
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

See "str.removeprefix()" for a method that will remove a single prefix string rather than all of a set of characters. For example:

```
>>> 'Arthur: three!'.lstrip('Arthur: ')
'ee!'
>>> 'Arthur: three!'.removeprefix('Arthur: ')
'three!'
```

`static str.maketrans(dict, /)`

`static str.maketrans(from, to, remove="", /)`

This static method returns a translation table usable for "str.translate()".

If there is only one argument, it must be a dictionary mapping Unicode ordinals (integers) or characters (strings of length 1) to Unicode ordinals, strings (of arbitrary lengths) or "None". Character keys will then be converted to ordinals.

If there are two arguments, they must be strings of equal length, and in the resulting dictionary, each character in **from** will be mapped to the character at the same position in **to**. If there is a third argument, it must be a string, whose characters will be mapped to "None" in the result.

`str.partition(sep, /)`

Split the string at the first occurrence of **sep**, and return a

3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing the string itself, followed by two empty strings.

For example:

```
>>> 'Monty Python'.partition(' ')
('Monty', ' ', 'Python')
>>> "Monty Python's Flying Circus".partition(' ')
('Monty', ' ', "Python's Flying Circus")
>>> 'Monty Python'.partition('-')
('Monty Python', '-', '')
```

See also `rrpartition()`.

`str.removeprefix(prefix, /)`

If the string starts with the *prefix* string, return `string[len(prefix):]`. Otherwise, return a copy of the original string:

```
>>> 'TestHook'.removeprefix('Test')
'Hook'
>>> 'BaseTestCase'.removeprefix('Test')
'BaseTestCase'
```

Added in version 3.9.

See also `removesuffix()` and `startswith()`.

`str.removesuffix(suffix, /)`

If the string ends with the *suffix* string and that *suffix* is not empty, return `string[:-len(suffix)]`. Otherwise, return a copy of the original string:

```
>>> 'MiscTests'.removesuffix('Tests')
'Misc'
>>> 'TmpDirMixin'.removesuffix('Tests')
'TmpDirMixin'
```

Added in version 3.9.

See also `removeprefix()` and `endswith()`.

`str.replace(old, new, /, count=-1)`

Return a copy of the string with all occurrences of substring *old* replaced by *new*. If *count* is given, only the first *count* occurrences are replaced. If *count* is not specified or `-1`, then all occurrences are replaced. For example:

```
>>> 'spam, spam, spam'.replace('spam', 'eggs')
'eggs, eggs, eggs'
>>> 'spam, spam, spam'.replace('spam', 'eggs', 1)
'eggs, spam, spam'
```

Changed in version 3.13: *count* is now supported as a keyword argument.

`str.rfind(sub[, start[, end]])`

Return the highest index in the string where substring **sub** is found, such that **sub** is contained within *"s[start:end]"*. Optional arguments **start** and **end** are interpreted as in slice notation. Return *"-1"* on failure. For example:

```
>>> 'spam, spam, spam'.rfind('sp')
12
>>> 'spam, spam, spam'.rfind('sp', 0, 10)
6
```

See also *"find()"* and *"rindex()"*.

str.rindex(sub[, start[, end]])

Like *"rfind()"* but raises *"ValueError"* when the substring **sub** is not found. For example:

```
>>> 'spam, spam, spam'.rindex('spam')
12
>>> 'spam, spam, spam'.rindex('eggs')
Traceback (most recent call last):
  File "<stdin-0>", line 1, in <module>
    'spam, spam, spam'.rindex('eggs')
    ~~~~~~^~~~~~
ValueError: substring not found
```

See also *"index()"* and *"find()"*.

str.rjust(width, fillchar=' ', /)

Return the string right justified in a string of length **width**. Padding is done using the specified **fillchar** (default is an ASCII space). The original string is returned if **width** is less than or equal to *"len(s)"*.

For example:

```
>>> 'Python'.rjust(10)
'  Python'
>>> 'Python'.rjust(10, '.')
'....Python'
>>> 'Monty Python'.rjust(10, '.')
'Monty Python'
```

See also *"ljust()"* and *"zfill()"*.

str.rpartition(sep, /)

Split the string at the last occurrence of **sep**, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing two empty strings, followed by the string itself.

For example:

```
>>> 'Monty Python'.rpartition(' ')
('Monty', ' ', 'Python')
>>> "Monty Python's Flying Circus".rpartition(' ')
('Monty Python's Flying', ' ', 'Circus')
>>> 'Monty Python'.rpartition('-')
```

```
("", " ", 'Monty Python')
```

See also "partition()".

```
str.rsplit(sep=None, maxsplit=-1)
```

Return a list of the words in the string, using **sep** as the delimiter string. If **maxsplit** is given, at most **maxsplit** splits are done, the **rightmost** ones. If **sep** is not specified or "None", any whitespace string is a separator. Except for splitting from the right, "rsplit()" behaves like "split()" which is described in detail below.

```
str.rstrip(chars=None, /)
```

Return a copy of the string with trailing characters removed. The **chars** argument is a string specifying the set of characters to be removed. If omitted or "None", the **chars** argument defaults to removing whitespace. The **chars** argument is not a suffix; rather, all combinations of its values are stripped. For example:

```
>>> ' spacious '.rstrip()
' spacious'
>>> 'mississippi'.rstrip('ipz')
'mississ'
```

See "removesuffix()" for a method that will remove a single suffix string rather than all of a set of characters. For example:

```
>>> 'Monty Python'.rstrip(' Python')
'M'
>>> 'Monty Python'.removesuffix(' Python')
'Monty'
```

See also "strip()".

```
str.split(sep=None, maxsplit=-1)
```

Return a list of the words in the string, using **sep** as the delimiter string. If **maxsplit** is given, at most **maxsplit** splits are done (thus, the list will have at most "**maxsplit*+1**" elements). If **maxsplit** is not specified or "-1", then there is no limit on the number of splits (all possible splits are made).

If **sep** is given, consecutive delimiters are not grouped together and are deemed to delimit empty strings (for example, "'1,,2'.split(',')" returns "['1', '', '2']"). The **sep** argument may consist of multiple characters as a single delimiter (to split with multiple delimiters, use "re.split()"). Splitting an empty string with a specified separator returns "[]".

For example:

```
>>> '1,2,3'.split(',')
['1', '2', '3']
>>> '1,2,3'.split(',', maxsplit=1)
['1', '2,3']
>>> '1,2,,3'.split(',')
['1', '2', '', '3', '']
>>> '1<>2<>3<4'.split('<>')
['1', '2', '3<4']
```

If **sep** is not specified or is "None", a different splitting algorithm is applied: runs of consecutive whitespace are regarded as a single separator, and the result will contain no empty strings at the start or end if the string has leading or trailing whitespace. Consequently, splitting an empty string or a string consisting of just whitespace with a "None" separator returns [].

For example:

```
>>> '1 2 3'.split()
['1', '2', '3']
>>> '1 2 3'.split(maxsplit=1)
['1', '2 3']
>>> ' 1 2 3 '.split()
['1', '2', '3']
```

If **sep** is not specified or is "None" and **maxsplit** is "0", only leading runs of consecutive whitespace are considered.

For example:

```
>>> "".split(None, 0)
[]
>>> " ".split(None, 0)
[]
>>> " foo ".split(maxsplit=0)
['foo ']
```

See also "join()" and "rsplit()".

str.splitlines(keepends=False)

Return a list of the lines in the string, breaking at line boundaries. Line breaks are not included in the resulting list unless **keepends** is given and true.

This method splits on THE following line boundaries. In [rev pdf2] particular, the boundaries are a superset of **universal newlines**.

Representation	Description
"\n"	Line Feed
"\r"	Carriage Return
"\r\n"	Carriage Return + Line Feed
"\v" or "\x0b"	Line Tabulation
"\f" or "\x0c"	Form Feed
"\x1c"	File Separator
"\x1d"	Group Separator
"\x1e"	Record Separator
"\x85"	Next Line (C1 Control Code)
"\u2028"	Line Separator

Changed in version 3.2: "\v" and "\f" added to list of line boundaries.

For example:

```
>>> 'ab c\n\nde fg\rkl\r\n'.splitlines()
['ab c', '', 'de fg', 'kl']
>>> 'ab c\n\nde fg\rkl\r\n'.splitlines(keepends=True)
['ab c\n', '\n', 'de fg\r', 'kl\r\n']
```

Unlike "split()" when a delimiter string **sep** is given, this method returns an empty list for the empty string, and a terminal line break does not result in an extra line:

```
>>> ''.splitlines()
[]
>>> "One line\n".splitlines()
['One line']
```

For comparison, "split('\n')" gives:

```
>>> ".split('\n')
['']
>>> 'Two lines\n'.split('\n')
['Two lines', '']
```

`str.startswith(prefix[, start[, end]])`

Return "True" if string starts with the **prefix**, otherwise return "False". **prefix** can also be a tuple of prefixes to look for. With optional **start**, test string beginning at that position. With optional **end**, stop comparing string at that position.

For example:

```
>>> 'Python'.startswith('Py')
True
>>> 'a tuple of prefixes'.startswith(('at', 'a'))
True
>>> 'Python is amazing'.startswith('is', 7)
True
```

See also "endswith()" and "removeprefix()".

`str.strip(chars=None, /)`

Return a copy of the string with the leading and trailing characters removed. The **chars** argument is a string specifying the set of characters to be removed. If omitted or "None", the **chars** argument defaults to removing whitespace. The **chars** argument is not a prefix or suffix; rather, all combinations of its values are stripped.

Whitespace characters are defined by "str.isspace()".

For example:

```
>>> '  spacious  '.strip()
'spacious'
>>> 'www.example.com'.strip('cmowz.')
'www.example.com'
```

'example'

The outermost leading and trailing **chars** argument values are stripped from the string. Characters are removed from the leading end until reaching a string character that is not contained in the set of characters in **chars**. A similar action takes place on the trailing end.

For example:

```
>>> comment_string = '#..... Section 3.2.1 Issue #32 .....'
>>> comment_string.strip('#! ')
'Section 3.2.1 Issue #32'
```

See also "rstrip()".

str.swapcase()

Return a copy of the string with uppercase characters converted to lowercase and vice versa. For example:

```
>>> 'Hello World'.swapcase()
'hELLO wORLD'
```

Note that it is not necessarily true that "s.swapcase().swapcase() == s". For example:

```
>>> 'straße'.swapcase().swapcase()
'strasse'
```

See also "str.lower()" and "str.upper()".

str.title()

Return a titlecased version of the string where words start with an uppercase character and the remaining characters are lowercase.

For example:

```
>>> 'Hello world'.title()
'Hello World'
```

The algorithm uses a simple language-independent definition of a word as groups of consecutive letters. The definition works in many contexts but it means that apostrophes in contractions and possessives form word boundaries, which may not be the desired result:

```
>>> "they're bill's friends from the UK".title()
'They'Re Bill'S Friends From The Uk'
```

The "string.capwords()" function does not have this problem, as it splits words on spaces only.

Alternatively, a workaround for apostrophes can be constructed using regular expressions:

```
>>> import re
>>> def titlecase(s):
...     return re.sub(r"[A-Za-z]+('[A-Za-z]+)?",
...                   lambda mo: mo.group(0).capitalize(),
...                   s)
```

```
...
>>> titlecase("they're bill's friends.")
'They're Bill's Friends.'
```

See also "istitle()".

`str.translate(table, /)`

Return a copy of the string in which each character has been mapped through the given translation table. The table must be an object that implements indexing via "`__getitem__()`", typically a **mapping** or **sequence**. When indexed by a Unicode ordinal (an integer), the table object can do any of the following: return a Unicode ordinal or a string, to map the character to one or more other characters; return "None", to delete the character from the return string; or raise a "LookupError" exception, to map the character to itself.

You can use "`str.maketrans()`" to create a translation map from character-to-character mappings in different formats.

See also the "codecs" module for a more flexible approach to custom character mappings.

`str.upper()`

Return a copy of the string with all the cased characters [4] converted to uppercase. Note that "`s.upper().isupper()`" might be "False" if "s" contains uncased characters or if the Unicode category of the resulting character(s) is not `·Lu·` (Letter, uppercase), but e.g. `·Lt·` (Letter, titlecase).

The uppercasing algorithm used is described in section 3.13 `·Default Case Folding·` of the Unicode Standard.

`str.zfill(width, /)`

Return a copy of the string left filled with ASCII `"0"` digits to make a string of length **width**. A leading sign prefix (`"+" / "-"`) is handled by inserting the padding **after** the sign character rather than before. The original string is returned if **width** is less than or equal to "`len(s)`".

For example:

```
>>> "42".zfill(5)
'00042'
>>> "-42".zfill(5)
'-0042'
```

See also "`rjust()`".

STRINGS

=====

String and Bytes literals

String literals are text enclosed in single quotes (`"`) or double quotes (`"`). For example:

```
"spam"
```

'eggs'

The quote used to start the literal also terminates it, so a string literal can only contain the other quote (except with escape sequences, see below). For example:

```
'Say "Hello", please.'  
"Don't do that!"
```

Except for this limitation, the choice of quote character ("" or """) does not affect how the literal is parsed.

Inside a string literal, the backslash ("\") character introduces an *escape sequence*, which has special meaning depending on the character after the backslash. For example, "\"" denotes the double quote character, and does *not* end the string:

```
>>> print("Say \"Hello\" to everyone!")  
Say "Hello" to everyone!
```

See escape sequences below for a full list of such sequences, and more details.

Triple-quoted strings

=====

Strings can also be enclosed in matching groups of three single or double quotes. These are generally referred to as *triple-quoted strings*:

```
"""This is a triple-quoted string."""
```

In triple-quoted literals, unescaped quotes are allowed (and are retained), except that three unescaped quotes in a row terminate the literal, if they are of the same kind ("" or """) used at the start:

```
"""This string has "quotes" inside."""
```

Unescaped newlines are also allowed and retained:

```
"""This triple-quoted string  
continues on the next line."""
```

String prefixes

=====

String literals can have an optional *prefix* that influences how the content of the literal is parsed, for example:

```
b"data"  
f{result=}
```

The allowed prefixes are:

* "b": Bytes literal

* "r": Raw string

* "f": Formatted string literal (.f-string.)

- * "t": Template string literal (·t-string·)
- * "u": No effect (allowed for backwards compatibility)

See the linked sections for details on each type.

Prefixes are case-insensitive (for example, ·"B"· works the same as ·"b"·). The ·"r"· prefix can be combined with ·"f"·, ·"t"· or ·"b"·, so ·"fr"·, ·"rf"·, ·"tr"·, ·"rt"·, ·"br"·, and ·"rb"· are also valid
 NOTE (pdf2): paragraph added at revision, line 1371.
 prefixes.

Added in version 3.3: The ""rb"" prefix of raw bytes literals has been added as a synonym of ""br"".Support for the unicode legacy literal ("u'value'") was reintroduced to simplify the maintenance of dual Python 2.x and 3.x codebases. See ****PEP 414**** for more information.

Formal grammar

=====

String literals, except ·f-strings· and ·t-strings·, are described by the following lexical definitions.

These definitions use negative lookaheads ("!") to indicate that an ending quote ends the literal.

```

STRING:      [stringprefix] (stringcontent)
stringprefix: <("r" | "u" | "b" | "br" | "rb"), case-insensitive>
stringcontent:
  | """" ( !"""" longstringitem)* """"
  | """" ( !"""" longstringitem)* """"
  | "" ( !"" stringitem)* ""
  | "" ( !"" stringitem)* ""
stringitem:   stringchar | stringescapeseq
stringchar:   <any source_character, except backslash and newline>
longstringitem: stringitem | newline
stringescapeseq: "\" <any source_character>
```

Note that as in all lexical definitions, whitespace is significant. In particular, the prefix (if any) must be immediately followed by the starting quote.

Escape sequences

=====

Unless an ·"r"· or ·"R"· prefix is present, escape sequences in string and bytes literals are interpreted according to rules similar to those used by Standard C. The recognized escape sequences are:

+-----+-----	
-----+	
Escape Sequence	Meaning
===== =====	
===== \"<newline>	Ignored end of line
+-----+-----	
-----+	
\"	Backslash

-----+	
"\'"	Single quote
-----+	
"\"	Double quote
-----+	
"\a"	ASCII Bell (BEL)
-----+	
"\b"	ASCII Backspace (BS)
-----+	
"\f"	ASCII Formfeed (FF)
-----+	
"\n"	ASCII Linefeed (LF)
-----+	
"\r"	ASCII Carriage Return (CR)
-----+	
-----+	
-----+	
"\v"	ASCII Vertical Tab (VT)
-----+	
"*ooo*"	Octal character
-----+	
"\x*hh*"	Hexadecimal character
-----+	
"\N{*name*}"	Named Unicode character
-----+	
"\u*xxxx*"	Hexadecimal Unicode character
-----+	
"\U*xxxxxxxx*"	Hexadecimal Unicode character
-----+	

Ignored end of line

A backslash can be added at the end of a line to ignore the newline:

```
>>> 'This string will not include \
... backslashes or newline characters.'
'This string will not include backslashes or newline characters.'
```

The same result can be achieved using triple-quoted strings, or parentheses and string literal concatenation.

Escaped characters

To include a backslash in a non-raw Python string literal, it must be doubled. The `"\\`" escape sequence denotes a single backslash character:

```
>>> print('C:\\Program Files')
C:\Program Files
```

Similarly, the `"\'`" and `"\""` sequences denote the single and double quote character, respectively:

```
>>> print('\ ' and '\ ')
' and "
```

Octal character

The sequence `"\ooo"` denotes a *character* with the octal (base 8) value *ooo*:

```
>>> '\120'
'P'
```

Up to three octal digits (0 through 7) are accepted.

In a bytes literal, *character* means a *byte* with the given value. In a string literal, it means a Unicode character with the given value.

Changed in version 3.11: Octal escapes with value larger than `"0o377"` (255) produce a `"DeprecationWarning"`.

Changed in version 3.12: Octal escapes with value larger than `"0o377"` (255) produce a `"SyntaxWarning"`. In a future Python version they will raise a `"SyntaxError"`.

Hexadecimal character

The sequence `"\x*hh"` denotes a *character* with the hex (base 16) value *hh*:

```
>>> '\x50'
'P'
```

Unlike in Standard C, exactly two hex digits are required.

In a bytes literal, *character* means a *byte* with the given value.

In a string literal, it means a Unicode character with the given value.

Named Unicode character

The sequence `"\N{*name}"` denotes a Unicode character with the given `*name*`:

```
>>> '\N{LATIN CAPITAL LETTER P}'
'P'
>>> '\N{SNAKE}'
','
```

This sequence cannot appear in bytes literals.

Changed in version 3.3: Support for name aliases has been added.

Hexadecimal Unicode characters

These sequences `"\u*xxx*"` and `"\U*xxxxxxx*"` denote the Unicode character with the given hex (base 16) value. Exactly four digits are required for `"\u"`; exactly eight digits are required for `"\U"`. The latter can encode any Unicode character.

```
>>> '\u1234'
','
>>> '\U0001f40d'
','
```

These sequences cannot appear in bytes literals.

Unrecognized escape sequences

Unlike in Standard C, all unrecognized escape sequences are left in the string unchanged, that is, `*the backslash is left in the result*`:

```
>>> print('\q')
\q
>>> list('\q')
['\\', 'q']
```

Note that for bytes literals, the escape sequences only recognized in string literals (`"\N..."`, `"\u..."`, `"\U..."`) fall into the category of unrecognized escapes.

Changed in version 3.6: Unrecognized escape sequences produce a `"DeprecationWarning"`.

Changed in version 3.12: Unrecognized escape sequences produce a `"SyntaxWarning"`. In a future Python version they will raise a `"SyntaxError"`.

Bytes literals

=====

Bytes literals are always prefixed with `·"b"·` or `·"B"·`; they produce an instance of the `"bytes"` type instead of the `"str"` type. They may only contain ASCII characters; bytes with a numeric value of 128 or greater must be expressed with escape sequences (typically Hexadecimal character or Octal character):

```
>>> b'\x89PNG\r\n\x1a\n'
b'\x89PNG\r\n\x1a\n'
>>> list(b'\x89PNG\r\n\x1a\n')
[137, 80, 78, 71, 13, 10, 26, 10]
```

Similarly, a zero byte must be expressed using an escape sequence (typically `"\0"` or `"\x00"`).

Raw string literals

=====

Both string and bytes literals may optionally be prefixed with a letter `·"r"·` or `·"R"·`; such constructs are called **raw string literals** and **raw bytes literals** respectively and treat backslashes as literal characters. As a result, in raw string literals, escape sequences are not treated specially:

```
>>> r'\d{4}-\d{2}-\d{2}' [rev pdf2]
'\d{4}-\d{2}-\d{2}'
```

Even in a raw literal, quotes can be escaped with a backslash, but the backslash remains in the result; for example, `"r\""` is a valid string literal consisting of two characters: a backslash and a double quote; `"r\""` is not a valid string literal (even a raw string cannot end in an odd number of backslashes). Specifically, **a raw literal cannot end in a single backslash** (since the backslash would escape the following quote character). Note also that a single backslash followed by a newline is interpreted as those two characters as part of the literal, **not** as a line continuation.

f-strings

=====

Added in version 3.6.

Changed in version 3.7: The `"await"` and `"async for"` can be used in expressions within f-strings.

Changed in version 3.8: Added the debug specifier (`"=`)

Changed in version 3.12: Many restrictions on expressions within f-strings have been removed. Notably, nested strings, comments, and backslashes are now permitted.

A **formatted string literal** or **f-string** is a string literal that is prefixed with `·"f"·` or `·"F"·`. Unlike other string literals, f-strings do not have a constant value. They may contain **replacement fields** delimited by curly braces `"{}"`. Replacement fields contain expressions which are evaluated at run time. For example:

```
>>> who = 'nobody'
>>> nationality = 'Spanish'
>>> f'{who.title()} expects the {nationality} Inquisition!'
'Nobody expects the Spanish Inquisition!'
```

Any doubled curly braces ("{" or "}") outside replacement fields are replaced with the corresponding single curly brace:

```
>>> print(f'{{...}}')
{...}
```

Other characters outside replacement fields are treated like in ordinary string literals. This means that escape sequences are decoded (except when a literal is also marked as a raw string), and newlines are possible in triple-quoted f-strings:

```
>>> name = 'Galahad'
>>> favorite_color = 'blue'
>>> print(f'{name}:\t{favorite_color}')
Galahad:      blue
>>> print(rf"C:\Users\{name}")
C:\Users\Galahad
>>> print(f"""Three shall be the number of the counting
... and the number of the counting shall be three.""")
Three shall be the number of the counting
and the number of the counting shall be three.
```

Expressions in formatted string literals are treated like regular Python expressions. Each expression is evaluated in the context where the formatted string literal appears, in order from left to right. An empty expression is not allowed, and both "lambda" and assignment expressions ":"=" must be surrounded by explicit parentheses:

```
>>> f'{{(half := 1/2)}, {half * 42}}'
'0.5, 21.0'
```

Reusing the outer f-string quoting type inside a replacement field is permitted:

```
>>> a = dict(x=2)
>>> f"abc {a['x']} def"
'abc 2 def'
```

Backslashes are also allowed in replacement fields and are evaluated the same way as in any other context:

```
>>> a = ["a", "b", "c"]
>>> print(f"List a contains:\n{a}\n".join(a))
List a contains:
a
b
c
```

It is possible to nest f-strings:

```
>>> name = 'world'
>>> f'Repeated:{f' hello {name}' * 3}'
'Repeated: hello world hello world hello world'
```

Portable Python programs should not use more than 5 levels of nesting.

****CPython implementation detail:**** CPython does not limit nesting of f-strings.

Replacement expressions can contain newlines in both single-quoted and triple-quoted f-strings and they can contain comments. Everything that

comes after a "#" inside a replacement field is a comment (even closing braces and quotes). This means that replacement fields with comments must be closed in a different line:

```
>>> a = 2
>>> f"abc{a # This comment }" continues until the end of the line
...   + 3}"
'abc5'
```

After the expression, replacement fields may optionally contain:

- * a **debug specifier** · an equal sign ("="), optionally surrounded by whitespace on one or both sides;

- * a **conversion specifier** · "l", "r" or "a"; and/or

- * a **format specifier** prefixed with a colon (":").

See the Standard Library section on f-strings for details on how these fields are evaluated.

As that section explains, **format specifiers** are passed as the second argument to the "format()" function to format a replacement field value. For example, they can be used to specify a field width and padding characters using the Format Specification Mini-Language:

```
>>> number = 14.3
>>> f'{number:20.7f}'
'      14.3000000'
```

Top-level format specifiers may include nested replacement fields:

```
>>> field_size = 20
>>> precision = 7
>>> f'{number:{field_size}.{precision}f}'
'      14.3000000'
```

These nested fields may include their own conversion fields and format specifiers:

```
>>> number = 3
>>> f'{number:{field_size}}'
'          3'
>>> f'{number:{field_size:05}}'
'00000000000000000003'
```

However, these nested fields may not include more deeply nested replacement fields.

Formatted string literals cannot be used as **docstrings**, even if they do not include expressions:

```
>>> def foo():
...     f"Not a docstring"
...
>>> print(foo.__doc__)
None
```

See also:

- * **PEP 498** · Literal String Interpolation

* **PEP 701** · Syntactic formalization of f-strings

* "str.format()", which uses a related format string mechanism.

t-strings

=====

Added in version 3.14.

A *template string literal* or *t-string* is a string literal that is prefixed with `"t"` or `"T"`. These strings follow the same syntax rules as formatted string literals. For differences in evaluation rules, see the Standard Library section on t-strings

Formal grammar for f-strings

=====

F-strings are handled partly by the *lexical analyzer*, which produces the tokens `"FSTRING_START"`, `"FSTRING_MIDDLE"` and `"FSTRING_END"`, and partly by the parser, which handles expressions in the replacement field. The exact way the work is split is a CPython implementation detail.

Correspondingly, the f-string grammar is a mix of lexical and syntactic definitions.

Whitespace is significant in these situations:

* There may be no whitespace in `"FSTRING_START"` (between the prefix and quote).

* Whitespace in `"FSTRING_MIDDLE"` is part of the literal string contents.

* In `"fstring_replacement_field"`, if `"f_debug_specifier"` is present, all whitespace after the opening brace until the `"f_debug_specifier"`, as well as whitespace immediately following `"f_debug_specifier"`, is retained as part of the expression.

CPython implementation detail: The expression is not handled in the tokenization phase; it is retrieved from the source code using locations of the `"{"` token and the token after `"="`.

The `"FSTRING_MIDDLE"` definition uses negative lookaheads (`"!"`) to indicate special characters (backslash, newline, `"{"`, `"}"`) and sequences (`"f_quote"`).

fstring: FSTRING_START fstring_middle* FSTRING_END

FSTRING_START: fstringprefix ("" | "" | "" | "")

FSTRING_END: f_quote

fstringprefix: <("f" | "fr" | "rf"), case-insensitive>

f_debug_specifier: '='

f_quote: <the quote character(s) used in FSTRING_START>

fstring_middle:

| fstring_replacement_field

| FSTRING_MIDDLE

FSTRING_MIDDLE:

| (!" !newline !" !' !f_quote) source_character

```

| stringescape
| "{"
| "}"
| <newline, in triple-quoted f-strings only>
fstring_replacement_field:
| '{ f_expression [f_debug_specifier] [fstring_conversion]
  [fstring_full_format_spec] }'
fstring_conversion:
| "!" ("s" | "r" | "a")
fstring_full_format_spec:
| ':' fstring_format_spec*
fstring_format_spec:
| FString_MIDDLE
| fstring_replacement_field
f_expression:
| ',',(conditional_expression | "*" or_expr)+ [","]
| yield_expression

```

Note:

In the above grammar snippet, the "f_quote" and "FSTRING_MIDDLE" rules are context-sensitive · they depend on the contents of "FSTRING_START" of the nearest enclosing "fstring". Constructing a more traditional formal grammar from this template is left as an exercise for the reader.

The grammar for t-strings is identical to the one for f-strings, with *t* instead of *f* at the beginning of rule and token names and in the prefix.

```
tstring:  TSTRING_START tstring_middle* TSTRING_END
```

<rest of the t-string grammar is omitted; see above>

SUBSCRIPTIONS

=====

Subscriptions and slicings

The *subscription* syntax is usually used for selecting an element from a container · for example, to get a value from a "dict":

```

>>> digits_by_name = {'one': 1, 'two': 2}
>>> digits_by_name['two'] # Subscripting a dictionary using the key 'two'
2

```

In the subscription syntax, the object being subscribed · a primary · is followed by a *subscript* in square brackets. In the simplest case, the subscript is a single expression.

Depending on the type of the object being subscribed, the subscript is sometimes called a *key* (for mappings), *index* (for sequences), or *type argument* (for *generic types*). Syntactically, these are all equivalent:

```

>>> colors = ['red', 'blue', 'green', 'black']
>>> colors[3] # Subscripting a list using the index 3
'black'

>>> list[str] # Parameterizing the list type using the type argument str

```

```
list[str]
```

At runtime, the interpreter will evaluate the primary and the subscript, and call the primary's "`__getitem__()`" or "`__class__.__getitem__()`" *special method* with the subscript as argument. For more details on which of these methods is called, see `__class__.__getitem__` versus `__getitem__`.

To show how subscription works, we can define a custom object that implements "`__getitem__()`" and prints out the value of the subscript:

```
>>> class SubscriptionDemo:
...     def __getitem__(self, key):
...         print(f'subscripted with: {key!r}')
...
>>> demo = SubscriptionDemo()
>>> demo[1]
subscripted with: 1
>>> demo['a' * 3]
subscripted with: 'aaa'
```

See "`__getitem__()`" documentation for how built-in types handle subscription.

Subscriptions may also be used as targets in assignment or deletion statements. In these cases, the interpreter will call the subscripted object's "`__setitem__()`" or "`__delitem__()`" *special method*, respectively, instead of "`__getitem__()`".

```
>>> colors = ['red', 'blue', 'green', 'black']
>>> colors[3] = 'white' # Setting item at index
>>> colors
['red', 'blue', 'green', 'white']
>>> del colors[3] # Deleting item at index 3
>>> colors
['red', 'blue', 'green']
```

All advanced forms of *subscript* documented in the following sections are also usable for assignment and deletion.

Slicings

=====

A more advanced form of subscription, *slicing*, is commonly used to extract a portion of a sequence. In this form, the subscript is a *slice*: up to three expressions separated by colons. Any of the expressions may be omitted, but a slice must contain at least one colon:

```
>>> number_names = ['zero', 'one', 'two', 'three', 'four', 'five']
>>> number_names[1:3]
['one', 'two']
>>> number_names[1:]
['one', 'two', 'three', 'four', 'five']
>>> number_names[:3]
['zero', 'one', 'two']
>>> number_names[:]
['zero', 'one', 'two', 'three', 'four', 'five']
>>> number_names[::2]
['zero', 'two', 'four']
>>> number_names[:-3]
['zero', 'one', 'two']
```

```

['zero', 'one', 'two']
>>> del number_names[4:]
>>> number_names
['zero', 'one', 'two', 'three']

```

When a slice is evaluated, the interpreter constructs a "slice" object whose "start", "stop" and "step" attributes, respectively, are the results of the expressions between the colons. Any missing expression evaluates to "None". This "slice" object is then passed to the "`__getitem__()`" or "`__class_getitem__()`" *special method*, as above.

```

# continuing with the SubscriptionDemo instance defined above:
>>> demo[2:3]
subscripted with: slice(2, 3, None)
>>> demo[:, 'spam']
subscripted with: slice(None, None, 'spam')

```

Comma-separated subscripts

```
=====
```

The subscript can also be given as two or more comma-separated expressions or slices:

```

# continuing with the SubscriptionDemo instance defined above:
>>> demo[1, 2, 3]
subscripted with: (1, 2, 3)
>>> demo[1:2, 3]
subscripted with: (slice(1, 2, None), 3)

```

This form is commonly used with numerical libraries for slicing multi-dimensional data. In this case, the interpreter constructs a "tuple" of the results of the expressions or slices, and passes this tuple to the "`__getitem__()`" or "`__class_getitem__()`" *special method*, as above.

The subscript may also be given as a single expression or slice followed by a comma, to specify a one-element tuple:

```

>>> demo['spam',]
subscripted with: ('spam',)

```

Starred subscriptions

```
=====
```

Added in version 3.11: Expressions in *tuple_slices* may be starred. See **PEP 646**.

The subscript can also contain a starred expression. In this case, the interpreter unpacks the result into a tuple, and passes this tuple to "`__getitem__()`" or "`__class_getitem__()`":

```

# continuing with the SubscriptionDemo instance defined above:
>>> demo[*range(10)]
subscripted with: (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

```

Starred expressions may be combined with comma-separated expressions and slices:

```

>>> demo['a', 'b', *range(3), 'c']
subscripted with: ('a', 'b', 0, 1, 2, 'c')

```

Formal subscription grammar

=====

```
subscription:  primary '[' subscript ']'
subscript:     single_subscript | tuple_subscript
single_subscript: proper_slice | assignment_expression
proper_slice:   [expression] ":" [expression] [ ":" [expression] ]
tuple_subscript: ', '(single_subscript | starred_expression)+ [, ']
```

Recall that the "|" operator denotes ordered choice. Specifically, in "subscript", if both alternatives would match, the first ("single_subscript") has priority.

TRUTH

=====

Truth Value Testing

Any object can be tested for truth value, for use in an "if" or "while" condition or as operand of the Boolean operations below.

By default, an object is considered true unless its class defines either a "__bool__()" method that returns "False" or a "__len__()" method that returns zero, when called with the object. [1] If one of the methods raises an exception when called, the exception is propagated and the object does not have a truth value (for example, "NotImplemented"). Here are most of the built-in objects considered false:

* constants defined to be false: "None" and "False"

* zero of any numeric type: "0", "0.0", "0j", "Decimal(0)", "Fraction(0, 1)"

* empty sequences and collections: "", "()", "[]", "{}", "set()", "range(0)"

Operations and built-in functions that have a Boolean result always return "0" or "False" for false and "1" or "True" for true, unless otherwise stated. (Important exception: the Boolean operations "or" and "and" always return one of their operands.)

TRY

===

The "try" statement

The "try" statement specifies exception handlers and/or cleanup code for a group of statements:

```
try_stmt: try1_stmt | try2_stmt | try3_stmt
try1_stmt: "try" ":" suite
           ("except" [expression ["as" identifier]] ":" suite)+
           ["else" ":" suite]
           ["finally" ":" suite]
try2_stmt: "try" ":" suite
```

```

("except" "*" expression ["as" identifier] ":" suite)+
["else" ":" suite]
["finally" ":" suite]
try3_stmt: "try" ":" suite
          "finally" ":" suite

```

Additional information on exceptions can be found in section Exceptions, and information on using the "raise" statement to generate exceptions may be found in section The raise statement.

Changed in version 3.14: Support for optionally dropping grouping parentheses when using multiple exception types. See **PEP 758**.

```

"except" clause
=====

```

The "except" clause(s) specify one or more exception handlers. When no exception occurs in the "try" clause, no exception handler is executed. When an exception occurs in the "try" suite, a search for an exception handler is started. This search inspects the "except" clauses in turn until one is found that matches the exception. An expression-less "except" clause, if present, must be last; it matches any exception.

For an "except" clause with an expression, the expression must evaluate to an exception type or a tuple of exception types. Parentheses can be dropped if multiple exception types are provided and the "as" clause is not used. The raised exception matches an "except" clause whose expression evaluates to the class or a *non-virtual base class** of the exception object, or to a tuple that contains such a class.

If no "except" clause matches the exception, the search for an exception handler continues in the surrounding code and on the invocation stack. [1]

If the evaluation of an expression in the header of an "except" clause raises an exception, the original search for a handler is canceled and a search starts for the new exception in the surrounding code and on the call stack (it is treated as if the entire "try" statement raised the exception).

When a matching "except" clause is found, the exception is assigned to the target specified after the "as" keyword in that "except" clause, if present, and the "except" clause-s suite is executed. All "except" clauses must have an executable block. When the end of this block is reached, execution continues normally after the entire "try" statement. (This means that if two nested handlers exist for the same exception, and the exception occurs in the "try" clause of the inner handler, the outer handler will not handle the exception.)

When an exception has been assigned using "as target", it is cleared at the end of the "except" clause. This is as if:

```

except E as N:
    foo

```

was translated to:

```

except E as N:
    try:

```

```
foo
finally:
    del N
```

This means the exception must be assigned to a different name to be able to refer to it after the "except" clause. Exceptions are cleared because with the traceback attached to them, they form a reference cycle with the stack frame, keeping all locals in that frame alive until the next garbage collection occurs.

Before an "except" clause's suite is executed, the exception is stored in the "sys" module, where it can be accessed from within the body of the "except" clause by calling "sys.exception()". When leaving an exception handler, the exception stored in the "sys" module is reset to its previous value:

```
>>> print(sys.exception())
None
>>> try:
...     raise TypeError
... except:
...     print(repr(sys.exception()))
...     try:
...         raise ValueError
...     except:
...         print(repr(sys.exception()))
...         print(repr(sys.exception()))
...
TypeError()
ValueError()
TypeError()
>>> print(sys.exception())
None
```

"except*" clause
=====

The "except*" clause(s) specify one or more handlers for groups of exceptions ("BaseExceptionGroup" instances). A "try" statement can have either "except" or "except*" clauses, but not both. The exception type for matching is mandatory in the case of "except*", so "except*:" is a syntax error. The type is interpreted as in the case of "except", but matching is performed on the exceptions contained in the group that is being handled. A "TypeError" is raised if a matching type is a subclass of "BaseExceptionGroup", because that would have ambiguous semantics.

When an exception group is raised in the try block, each "except*" clause splits (see "split()") it into the subgroups of matching and non-matching exceptions. If the matching subgroup is not empty, it becomes the handled exception (the value returned from "sys.exception()") and assigned to the target of the "except*" clause (if there is one). Then, the body of the "except*" clause executes. If the non-matching subgroup is not empty, it is processed by the next "except*" in the same manner. This continues until all exceptions in the group have been matched, or the last "except*" clause has run.

After all "except*" clauses execute, the group of unhandled exceptions is merged with any exceptions that were raised or re-raised from within "except*" clauses. This merged exception group propagates on:

```

>>> try:
...     raise ExceptionGroup("eg",
...         [ValueError(1), TypeError(2), OSError(3), OSError(4)])
... except* TypeError as e:
...     print(f'caught {type(e)} with nested {e.exceptions}')
... except* OSError as e:
...     print(f'caught {type(e)} with nested {e.exceptions}')
...
caught <class 'ExceptionGroup'> with nested (TypeError(2),)
caught <class 'ExceptionGroup'> with nested (OSError(3), OSError(4))
+ Exception Group Traceback (most recent call last):
| File "<doctest default[0]>", line 2, in <module>
|   raise ExceptionGroup("eg",
|     [ValueError(1), TypeError(2), OSError(3), OSError(4)])
| ExceptionGroup: eg (1 sub-exception)
+-+----- 1 -----
| ValueError: 1
+-----

```

If the exception raised from the "try" block is not an exception group and its type matches one of the "except*" clauses, it is caught and wrapped by an exception group with an empty message string. This ensures that the type of the target "e" is consistently "BaseExceptionGroup":

```

>>> try:
...     raise BlockingIOError
... except* BlockingIOError as e:
...     print(repr(e))
...
ExceptionGroup("", (BlockingIOError(),))

```

"break", "continue" and "return" cannot appear in an "except*" clause.

"else" clause
=====

The optional "else" clause is executed if the control flow leaves the "try" suite, no exception was raised, and no "return", "continue", or "break" statement was executed. Exceptions in the "else" clause are not handled by the preceding "except" clauses.

"finally" clause
=====

If "finally" is present, it specifies a ·cleanup· handler. The "try" clause is executed, including any "except" and "else" clauses. If an exception occurs in any of the clauses and is not handled, the exception is temporarily saved. The "finally" clause is executed. If there is a saved exception it is re-raised at the end of the "finally" clause. If the "finally" clause raises another exception, the saved exception is set as the context of the new exception. If the "finally" clause executes a "return", "break" or "continue" statement, the saved exception is discarded. For example, this function returns 42.

```

def f():
    try:
        1/0
    finally:
        return 42

```

The exception information is not available to the program during execution of the "finally" clause.

When a "return", "break" or "continue" statement is executed in the "try" suite of a "try"-"finally" statement, the "finally" clause is also executed on the way out.

The return value of a function is determined by the last "return" statement executed. Since the "finally" clause always executes, a "return" statement executed in the "finally" clause will always be the last one executed. The following function returns "finally".

```
def foo():
    try:
        return 'try'
    finally:
        return 'finally'
```

Changed in version 3.8: Prior to Python 3.8, a "continue" statement was illegal in the "finally" clause due to a problem with the implementation.

Changed in version 3.14: The compiler emits a "SyntaxWarning" when a "return", "break" or "continue" appears in a "finally" block (see [**PEP 765**](#)).

TYPES

=====

The standard type hierarchy

Below is a list of the types that are built into Python. Extension modules (written in C, Java, or other languages, depending on the implementation) can define additional types. Future versions of Python may add types to the type hierarchy (e.g., rational numbers, efficiently stored arrays of integers, etc.), although such additions will often be provided via the standard library instead.

Some of the type descriptions below contain a paragraph listing special attributes. These are attributes that provide access to the implementation and are not intended for general use. Their definition may change in the future.

None

=====

This type has a single value. There is a single object with this value. This object is accessed through the built-in name "None". It is used to signify the absence of a value in many situations, e.g., it is returned from functions that don't explicitly return anything. Its truth value is false.

NotImplemented

=====

This type has a single value. There is a single object with this value. This object is accessed through the built-in name

"NotImplemented". Numeric methods and rich comparison methods should return this value if they do not implement the operation for the operands provided. (The interpreter will then try the reflected operation, or some oTHER fallback, depending on the operator.) It [rev pdf2] should not be evaluated in a boolean context.

See Implementing the arithmetic operations for more details.

Changed in version 3.9: Evaluating "NotImplemented" in a boolean context was deprecated.

Changed in version 3.14: Evaluating "NotImplemented" in a boolean context now raises a "TypeError". It previously evaluated to "True" and emitted a "DeprecationWarning" since Python 3.9.

Ellipsis
=====

This type has a single value. There is a single object with this value. This object is accessed through the literal "..." or the built-in name "Ellipsis". Its truth value is true.

"numbers.Number"
=====

These are created by numeric literals and returned as results by arithmetic operators and arithmetic built-in functions. Numeric objects are immutable; once created their value never changes. Python numbers are of course strongly related to mathematical numbers, but subject to the limitations of numerical representation in computers.

The string representations of the numeric classes, computed by "`__repr__()`" and "`__str__()`", have the following properties:

- * They are valid numeric literals which, when passed to their class constructor, produce an object having the value of the original numeric.
- * The representation is in base 10, when possible.
- * Leading zeros, possibly excepting a single zero before a decimal point, are not shown.
- * Trailing zeros, possibly excepting a single zero after a decimal point, are not shown.
- * A sign is shown only when the number is negative.

Python distinguishes between integers, floating-point numbers, and complex numbers:

"numbers.Integral"

These represent elements from the mathematical set of integers (positive and negative).

Note:

The rules for integer representation are intended to give the most meaningful interpretation of shift and mask operations involving negative integers.

There are two types of integers:

Integers ("int")

These represent numbers in an unlimited range, subject to available (virtual) memory only. For the purpose of shift and mask operations, a binary representation is assumed, and negative numbers are represented in a variant of 2's complement which gives the illusion of an infinite string of sign bits extending to the left.

Booleans ("bool")

These represent the truth values False and True. The two objects representing the values "False" and "True" are the only Boolean objects. The Boolean type is a subtype of the integer type, and Boolean values behave like the values 0 and 1, respectively, in almost all contexts, the exception being that when converted to a string, the strings ""False"" or ""True"" are returned, respectively.

"numbers.Real" ("float")

These represent machine-level double precision floating-point numbers. You are at the mercy of the underlying machine architecture (and C or Java implementation) for the accepted range and handling of overflow. Python does not support single-precision floating-point numbers; the savings in processor and memory usage that are usually the reason for using these are dwarfed by the overhead of using objects in Python, so there is no reason to complicate the language with two kinds of floating-point numbers.

"numbers.Complex" ("complex")

These represent complex numbers as a pair of machine-level double precision floating-point numbers. The same caveats apply as for floating-point numbers. The real and imaginary parts of a complex number "z" can be retrieved through the read-only attributes "z.real" and "z.imag".

Sequences

These represent finite ordered sets indexed by non-negative numbers. The built-in function "len()" returns the number of items of a sequence. When the length of a sequence is *n*, the index set contains the numbers 0, 1, ..., *n*-1. Item *i* of sequence *a* is selected by "a[i]". Some sequences, including built-in sequences, interpret negative subscripts by adding the sequence length. For example, "a[-2]" equals "a[n-2]", the second to last item of sequence a with length "n".

The resulting value must be a nonnegative integer less than the number of items in the sequence. If it is not, an "IndexError" is raised.

Sequences also support slicing: "a[start:stop]" selects all items with index *k* such that *start* " $\leq$ " *k* "<" *stop*. When used as an expression, a slice is a sequence of the same type. The comment above about negative subscripts also applies to negative slice positions. Note that no error is raised if a slice position is less than zero or larger than the length of the sequence.

If *start* is missing or "None", slicing behaves as if *start* was zero. If *stop* is missing or "None", slicing behaves as if *stop* was equal to the length of the sequence.

Some sequences also support ·extended slicing· with a third ·step· parameter: "a[i:j:k]" selects all items of *a* with index *x* where " $x = i + n * k$ ", *n* " $\geq$ " "0" and *j* " $\leq$ " *x* "<" *j*.

Sequences are distinguished according to their mutability:

Immutable sequences

An object of an immutable sequence type cannot change once it is created. (If the object contains references to other objects, these other objects may be mutable and may be changed; however, the collection of objects directly referenced by an immutable object cannot change.)

The following types are immutable sequences:

Strings

A string ("str") is a sequence of values that represent *characters*, or more formally, *Unicode code points*. All the code points in the range "0" to "0x10FFFF" can be represented in a string.

Python doesn't have a dedicated *character* type. Instead, every code point in the string is represented as a string object with length "1".

The built-in function "ord()" converts a code point from its string form to an integer in the range "0" to "0x10FFFF"; "chr()" converts an integer in the range "0" to "0x10FFFF" to the corresponding length "1" string object. "str.encode()" can be used to convert a "str" to "bytes" using the given text encoding, and "bytes.decode()" can be used to achieve the opposite.

Tuples

The items of a "tuple" are arbitrary Python objects. Tuples of two or more items are formed by comma-separated lists of expressions. A tuple of one item (a ·singleton·) can be formed by affixing a comma to an expression (an expression by itself does not create a tuple, since parentheses must be usable for grouping of expressions). An empty tuple can be formed by an empty pair of parentheses.

Bytes

A "bytes" object is an immutable array. The items are 8-bit bytes, represented by integers in the range $0 \leq x < 256$. Bytes literals (like "b'abc'") and the built-in "bytes()" constructor can be used to create bytes objects. Also, bytes objects can be decoded to

Mutable sequences

Mutable sequences can be changed after they are created. The subscription and slicing notations can be used as the target of assignment and "del" (delete) statements.

Note:

The "collections" and "array" module provide additional examples of mutable sequence types.

There are currently two intrinsic mutable sequence types:

Lists

The items of a list are arbitrary Python objects. Lists are formed by placing a comma-separated list of expressions in square brackets. (Note that there are no special cases needed to form lists of length 0 or 1.)

Byte Arrays

A bytearray object is a mutable array. They are created by the built-in "bytearray()" constructor. Aside from being mutable (and hence unhashable), byte arrays otherwise provide the same interface and functionality as immutable "bytes" objects.

Set types

=====

These represent unordered, finite sets of unique, immutable objects. As such, they cannot be indexed by any subscript. However, they can be iterated over, and the built-in function "len()" returns the number of items in a set. Common uses for sets are fast membership testing, removing duplicates from a sequence, and computing mathematical operations such as intersection, union, difference, and symmetric difference.

For set elements, the same immutability rules apply as for dictionary keys. Note that numeric types obey the normal rules for numeric comparison: if two numbers compare equal (e.g., "1" and "1.0"), only one of them can be contained in a set.

There are currently two intrinsic set types:

Sets

These represent a mutable set. They are created by the built-in "set()" constructor and can be modified afterwards by several methods, such as "add()".

Frozen sets

These represent an immutable set. They are created by the built-in "frozenset()" constructor. As a frozenset is immutable and *hashable*, it can be used again as an element of another set, or as a dictionary key.

Mappings

=====

These represent finite sets of objects indexed by arbitrary index sets. The subscript notation "a[k]" selects the item indexed by "k"

from the mapping "a"; this can be used in expressions and as the target of assignments or "del" statements. The built-in function "len()" returns the number of items in a mapping.

There is currently a single intrinsic mapping type:

Dictionaries

These represent finite sets of objects indexed by nearly arbitrary values. The only types of values not acceptable as keys are values containing lists or dictionaries or other mutable types that are compared by value rather than by object identity, the reason being that the efficient implementation of dictionaries requires a key's hash value to remain constant. Numeric types used for keys obey the normal rules for numeric comparison: if two numbers compare equal (e.g., "1" and "1.0") then they can be used interchangeably to index the same dictionary entry.

Dictionaries preserve insertion order, meaning that keys will be produced in the same order they were added sequentially over the dictionary. Replacing an existing key does not change the order, however removing a key and re-inserting it will add it to the end instead of keeping its old place.

Dictionaries are mutable; they can be created by the "{}" notation (see section Dictionary displays).

The extension modules "dbm.ndbm" and "dbm.gnu" provide additional examples of mapping types, as does the "collections" module.

Changed in version 3.7: Dictionaries did not preserve insertion order in versions of Python before 3.6. In CPython 3.6, insertion order was preserved, but it was considered an implementation detail at that time rather than a language guarantee.

Callable types
=====

These are the types to which the function call operation (see section Calls) can be applied:

User-defined functions

A user-defined function object is created by a function definition (see section Function definitions). It should be called with an argument list containing the same number of items as the function's formal parameter list.

Special read-only attributes
~~~~~

|              |         |
|--------------|---------|
| +-----+----- |         |
| -----+       |         |
| Attribute    | Meaning |
|              |         |
| ===== =====  |         |

```

=====|
| function.__builtins__          | A reference to the "dictionary" that
holds the |
|                               | function's builtins namespace. Added in
version |
|                               | 3.10.
|                               |
+-----+-----+

```

```

-----+
| function.__globals__          | A reference to the "dictionary" that
holds the |
|                               | function's global variables · the global
namespace |
|                               | of the module in which the function was
defined. |
+-----+-----+

```

```

-----+
| function.__closure__          | "None" or a "tuple" of cells that
contain bindings |
|                               | for the names specified in the
"co_freevars" |
|                               | attribute of the function's "code
object". A cell |
|                               | object has the attribute
"cell_contents". This can |
|                               | be used to get the value of the cell, as
well as |
|                               | set the value.
|                               |
+-----+-----+

```

## Special writable attributes

Most of these attributes check the type of the assigned value:

```

+-----+-----+
-----+
| Attribute                    | Meaning
|                               |
|=====|=====|
=====|
| function.__doc__            | The function's documentation string, or
"None" if |
|                               | unavailable.
|                               |
+-----+-----+
-----+
| function.__name__           | The function's name. See also: "__name__
|                               | attributes".
|                               |
+-----+-----+
-----+
| function.__qualname__       | The function's *qualified name*. See
also: |
|                               | "__qualname__ attributes". Added in
version 3.3. |
+-----+-----+
-----+

```

```

| function.__module__          | The name of the module the function was
defined |
|                             | in, or "None" if unavailable.
|
+-----+
-----+
| function.__defaults__       | A "tuple" containing default *parameter*
values |
|                             | for those parameters that have defaults,
or "None" |
|                             | if no parameters have a default value.
|
+-----+
-----+
| function.__code__           | The code object representing the
compiled function |
|                             | body.
|
+-----+
-----+
| function.__dict__           | The namespace supporting arbitrary
function |
|                             | attributes. See also: "__dict__
attributes". |
+-----+
-----+
| function.__annotations__     | A "dictionary" containing annotations of
|
|                             | *parameters*. The keys of the dictionary
are the |
|                             | parameter names, and "'return'" for the
return |
|                             | annotation, if provided. See also:
|
|                             | "object.__annotations__". Changed in
version |
|                             | 3.14: Annotations are now lazily
evaluated. See |
|                             | **PEP 649**.
|
+-----+
-----+
| function.__annotate__       | The *annotate function* for this
function, or |
|                             | "None" if the function has no
annotations. See |
|                             | "object.__annotate__". Added in version
3.14. |
+-----+
-----+
| function.__kwdefaults__     | A "dictionary" containing defaults for
keyword- |
|                             | only *parameters*.
|
+-----+
-----+
| function.__type_params__    | A "tuple" containing the type parameters
of a |
|                             | generic function. Added in version
3.12. |
+-----+
-----+

```

Function objects also support getting and setting arbitrary attributes, which can be used, for example, to attach metadata to functions. Regular attribute dot-notation is used to get and set such attributes.

**\*\*CPython implementation detail:\*\*** CPython-s current implementation only supports function attributes on user-defined functions. Function attributes on built-in functions may be supported in the future.

Additional information about a function-s definition can be retrieved from its code object (accessible via the "`__code__`" attribute).

### Instance methods

An instance method object combines a class, a class instance and any callable object (normally a user-defined function).

Special read-only attributes:

|                                 |                                                       |
|---------------------------------|-------------------------------------------------------|
| +-----+-----                    |                                                       |
| -----+                          |                                                       |
| method. <code>__self__</code>   | Refers to the class instance object to                |
| which the                       |                                                       |
|                                 | method is bound                                       |
|                                 |                                                       |
| +-----+-----                    |                                                       |
| -----+                          |                                                       |
| method. <code>__func__</code>   | Refers to the original function object                |
|                                 |                                                       |
| +-----+-----                    |                                                       |
| -----+                          |                                                       |
| method. <code>__doc__</code>    | The method-s documentation (same as                   |
|                                 |                                                       |
|                                 | " <code>method.__func__.__doc__</code> "). A "string" |
| if the                          |                                                       |
|                                 | original function had a docstring, else               |
| "None".                         |                                                       |
| +-----+-----                    |                                                       |
| -----+                          |                                                       |
| method. <code>__name__</code>   | The name of the method (same as                       |
|                                 |                                                       |
|                                 | " <code>method.__func__.__name__</code> ")            |
|                                 |                                                       |
| +-----+-----                    |                                                       |
| -----+                          |                                                       |
| method. <code>__module__</code> | The name of the module the method was                 |
| defined in,                     |                                                       |
|                                 | or "None" if unavailable.                             |
|                                 |                                                       |
| +-----+-----                    |                                                       |
| -----+                          |                                                       |

Methods also support accessing (but not setting) the arbitrary function attributes on the underlying function object.

User-defined method objects may be created when getting an attribute of a class (perhaps via an instance of that class), if that attribute is a user-defined function object or a "classmethod" object.

When an instance method object is created by retrieving a user-defined function object from a class via one of its instances, its "`__self__`" attribute is the instance, and the method object is said to be *\*bound\**. The new method's "`__func__`" attribute is the original function object.

When an instance method object is created by retrieving a "classmethod" object from a class or instance, its "`__self__`" attribute is the class itself, and its "`__func__`" attribute is the function object underlying the class method.

When an instance method object is called, the underlying function ("`__func__`") is called, inserting the class instance ("`__self__`") in front of the argument list. For instance, when "C" is a class which contains a definition for a function "f()", and "x" is an instance of "C", calling "x.f(1)" is equivalent to calling "C.f(x, 1)".

When an instance method object is derived from a "classmethod" object, the *\*class instance\** stored in "`__self__`" will actually be the class itself, so that calling either "x.f(1)" or "C.f(1)" is equivalent to calling "f(C,1)" where "f" is the underlying function.

It is important to note that user-defined functions which are attributes of a class instance are not converted to bound methods; this *\*only\** happens when the function is an attribute of the class.

#### Generator functions

-----

A function or method which uses the "yield" statement (see section The yield statement) is called a *\*generator function\**. Such a function, when called, always returns an *\*iterator\** object which can be used to execute the body of the function: calling the iterator's "`iterator.__next__()`" method will cause the function to execute until it provides a value using the "yield" statement. When the function executes a "return" statement or falls off the end, a "StopIteration" exception is raised and the iterator will have reached the end of the set of values to be returned.

#### Coroutine functions

-----

A function or method which is defined using "async def" is called a *\*coroutine function\**. Such a function, when called, returns a *\*coroutine\** object. It may contain "await" expressions, as well as "async with" and "async for" statements. See also the Coroutine Objects section.

#### Asynchronous generator functions

-----

A function or method which is defined using "async def" and which uses the "yield" statement is called a *\*asynchronous generator function\**. Such a function, when called, returns an *\*asynchronous iterator\** object which can be used in an "async for" statement to execute the body of the function.

Calling the asynchronous iterator's "`aiterator.__anext__`" method will return an *\*awaitable\** which when awaited will execute until it

provides a value using the "yield" expression. When the function executes an empty "return" statement or falls off the end, a "StopAsyncIteration" exception is raised and the asynchronous iterator will have reached the end of the set of values to be yielded.

## Built-in functions

-----

A built-in function object is a wrapper around a C function. Examples of built-in functions are "len()" and "math.sin()" ("math" is a standard built-in module). The number and type of the arguments are determined by the C function. Special read-only attributes:

\* "\_\_doc\_\_" is the function's documentation string, or "None" if unavailable. See "function.\_\_doc\_\_".

\* "\_\_name\_\_" is the function's name. See "function.\_\_name\_\_".

\* "\_\_self\_\_" is set to "None" (but see the next item).

\* "\_\_module\_\_" is the name of the module the function was defined in or "None" if unavailable. See "function.\_\_module\_\_".

## Built-in methods

-----

This is really a different disguise of a built-in function, this time containing an object passed to the C function as an implicit extra argument. An example of a built-in method is "alist.append()", assuming \*alist\* is a list object. In this case, the special read-only attribute "\_\_self\_\_" is set to the object denoted by \*alist\*. (The attribute has the same semantics as it does with "other instance methods".)

## Classes

-----

Classes are callable. These objects normally act as factories for new instances of themselves, but variations are possible for class types that override "\_\_new\_\_()". The arguments of the call are passed to "\_\_new\_\_()" and, in the typical case, to "\_\_init\_\_()" to initialize the new instance.

## Class Instances

-----

Instances of arbitrary classes can be made callable by defining a "\_\_call\_\_()" method in their class.

## Modules

=====

Modules are a basic organizational unit of Python code, and are created by the import system as invoked either by the "import" statement, or by calling functions such as "importlib.import\_module()" and built-in "\_\_import\_\_()". A module object has a namespace implemented by a "dictionary" object (this is the dictionary

referenced by the "`__globals__`" attribute of functions defined in the module). Attribute references are translated to lookups in this dictionary, e.g., "`m.x`" is equivalent to "`m.__dict__[\"x\"]`". A module object does not contain the code object used to initialize the module (since it isn't needed once the initialization is done).

Attribute assignment updates the module's namespace dictionary, e.g., "`m.x = 1`" is equivalent to "`m.__dict__[\"x\"] = 1`".

#### Import-related attributes on module objects

-----

Module objects have the following attributes that relate to the import system. When a module is created using the machinery associated with the import system, these attributes are filled in based on the module's `*spec*`, before the `*loader*` executes and loads the module.

To create a module dynamically rather than using the import system, it's recommended to use "`importlib.util.module_from_spec()`", which will set the various import-controlled attributes to appropriate values. It's also possible to use the "`types.ModuleType`" constructor to create modules directly, but this technique is more error-prone, as most attributes must be manually set on the module object after it has been created when using this approach.

#### Caution:

With the exception of "`__name__`", it is **strongly** recommended that you rely on "`__spec__`" and its attributes instead of any of the other individual attributes listed in this subsection. Note that updating an attribute on "`__spec__`" will not update the corresponding attribute on the module itself:

```
>>> import typing
>>> typing.__name__, typing.__spec__.name
('typing', 'typing')
>>> typing.__spec__.name = 'spelling'
>>> typing.__name__, typing.__spec__.name
('typing', 'spelling')
>>> typing.__name__ = 'keyboard_smashing'
>>> typing.__name__, typing.__spec__.name
('keyboard_smashing', 'spelling')
```

#### `module.__name__`

The name used to uniquely identify the module in the import system. For a directly executed module, this will be set to "`__main__`".

This attribute must be set to the fully qualified name of the module. It is expected to match the value of "`module.__spec__.name`".

#### `module.__spec__`

A record of the module's import-system-related state.

Set to the "module spec" that was used when importing the module. See Module specs for more details.

Added in version 3.4.

`module.__package__`

The *\*package\** a module belongs to.

If the module is top-level (that is, not a part of any specific package) then the attribute should be set to `""` (the empty string). Otherwise, it should be set to the name of the module's package (which can be equal to `"module.__name__"` if the module itself is a package). See **PEP 366** for further details.

This attribute is used instead of `"__name__"` to calculate explicit relative imports for main modules. It defaults to `"None"` for modules created dynamically using the `"types.ModuleType"` constructor; use `"importlib.util.module_from_spec()"` instead to ensure the attribute is set to a `"str"`.

It is **strongly** recommended that you use `"module.__spec__.parent"` instead of `"module.__package__"`. `"__package__"` is now only used as a fallback if `"__spec__.parent"` is not set, and this fallback path is deprecated.

Changed in version 3.4: This attribute now defaults to `"None"` for modules created dynamically using the `"types.ModuleType"` constructor. Previously the attribute was optional.

Changed in version 3.6: The value of `"__package__"` is expected to be the same as `"__spec__.parent"`. `"__package__"` is now only used as a fallback during import resolution if `"__spec__.parent"` is not defined.

Changed in version 3.10: `"ImportWarning"` is raised if an import resolution falls back to `"__package__"` instead of `"__spec__.parent"`.

Changed in version 3.12: Raise `"DeprecationWarning"` instead of `"ImportWarning"` when falling back to `"__package__"` during import resolution.

Deprecated since version 3.13, will be removed in version 3.15: `"__package__"` will cease to be set or taken into consideration by the import system or standard library.

`module.__loader__`

The *\*loader\** object that the import machinery used to load the module.

This attribute is mostly useful for introspection, but can be used for additional loader-specific functionality, for example getting data associated with a loader.

`"__loader__"` defaults to `"None"` for modules created dynamically using the `"types.ModuleType"` constructor; use `"importlib.util.module_from_spec()"` instead to ensure the attribute is set to a *\*loader\** object.

It is **strongly** recommended that you use `"module.__spec__.loader"` instead of `"module.__loader__"`.

Changed in version 3.4: This attribute now defaults to `"None"` for modules created dynamically using the `"types.ModuleType"` constructor. Previously the attribute was optional.

Deprecated since version 3.12, will be removed in version 3.16:  
Setting "`__loader__`" on a module while failing to set "`__spec__.loader`" is deprecated. In Python 3.16, "`__loader__`" will cease to be set or taken into consideration by the import system or the standard library.

`module.__path__`

A (possibly empty) \*sequence\* of strings enumerating the locations where the package's submodules will be found. Non-package modules should not have a "`__path__`" attribute. See `__path__` attributes on modules for more details.

It is **strongly** recommended that you use "`module.__spec__.submodule_search_locations`" instead of "`module.__path__`".

`module.__file__`

`module.__cached__`

"`__file__`" and "`__cached__`" are both optional attributes that may or may not be set. Both attributes should be a "str" when they are available.

"`__file__`" indicates the pathname of the file from which the module was loaded (if loaded from a file), or the pathname of the shared library file for extension modules loaded dynamically from a shared library. It might be missing for certain types of modules, such as C modules that are statically linked into the interpreter, and the import system may opt to leave it unset if it has no semantic meaning (for example, a module loaded from a database).

If "`__file__`" is set then the "`__cached__`" attribute might also be set, which is the path to any compiled version of the code (for example, a byte-compiled file). The file does not need to exist to set this attribute; the path can simply point to where the compiled file *would* exist (see **PEP 3147**).

Note that "`__cached__`" may be set even if "`__file__`" is not set. However, that scenario is quite atypical. Ultimately, the *\*loader\** is what makes use of the module spec provided by the *\*finder\** (from which "`__file__`" and "`__cached__`" are derived). So if a loader can load from a cached module but otherwise does not load from a file, that atypical scenario may be appropriate.

It is **strongly** recommended that you use "`module.__spec__.cached`" instead of "`module.__cached__`".

Deprecated since version 3.13, will be removed in version 3.15:  
Setting "`__cached__`" on a module while failing to set "`__spec__.cached`" is deprecated. In Python 3.15, "`__cached__`" will cease to be set or taken into consideration by the import system or standard library.

Other writable attributes on module objects

---

As well as the import-related attributes listed above, module objects also have the following writable attributes:

module.\_\_doc\_\_

The module's documentation string, or "None" if unavailable. See also: "\_\_doc\_\_" attributes".

module.\_\_annotations\_\_

A dictionary containing \*variable annotations\* collected during module body execution. For best practices on working with "\_\_annotations\_\_", see "annotationlib".

Changed in version 3.14: Annotations are now lazily evaluated. See **PEP 649**.

module.\_\_annotate\_\_

The \*annotate function\* for this module, or "None" if the module has no annotations. See also: "\_\_annotate\_\_" attributes.

Added in version 3.14.

## Module dictionaries

-----

Module objects also have the following special read-only attribute:

module.\_\_dict\_\_

The module's namespace as a dictionary object. Uniquely among the attributes listed here, "\_\_dict\_\_" cannot be accessed as a global variable from within a module; it can only be accessed as an attribute on module objects.

**CPython implementation detail:** Because of the way CPython clears module dictionaries, the module dictionary will be cleared when the module falls out of scope even if the dictionary still has live references. To avoid this, copy the dictionary or keep the module around while using its dictionary directly.

## Custom classes

=====

Custom class types are typically created by class definitions (see section Class definitions). A class has a namespace implemented by a dictionary object. Class attribute references are translated to lookups in this dictionary, e.g., "C.x" is translated to "C.\_\_dict\_\_['x']" (although there are a number of hooks which allow for other means of locating attributes). When the attribute name is not found there, the attribute search continues in the base classes. This search of the base classes uses the C3 method resolution order which behaves correctly even in the presence of ·diamond· inheritance structures where there are multiple inheritance paths leading back to a common ancestor. Additional details on the C3 MRO used by Python can be found at The Python 2.3 Method Resolution Order.

When a class attribute reference (for class "C", say) would yield a class method object, it is transformed into an instance method object whose "\_\_self\_\_" attribute is "C". When it would yield a "staticmethod" object, it is transformed into the object wrapped by

the static method object. See section Implementing Descriptors for another way in which attributes retrieved from a class may differ from those actually contained in its `"__dict__"`.

Class attribute assignments update the class's dictionary, never the dictionary of a base class.

A class object can be called (see above) to yield a class instance (see below).

Special attributes

| Attribute         | Meaning                                                                                                                                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| type.__name__     | The class's name. See also: <code>"__name__"</code> attributes.                                                                                                                                                       |
| type.__qualname__ | The class's *qualified name*. See also: <code>"__qualname__"</code> attributes.                                                                                                                                       |
| type.__module__   | The name of the module in which the class was defined.                                                                                                                                                                |
| type.__dict__     | A "mapping proxy" providing a read-only view of the class's namespace. See also: <code>"__dict__"</code> attributes.                                                                                                  |
| type.__bases__    | A "tuple" containing the class's bases. In most cases, for a class defined as <code>"class X(A, B, C)"</code> , <code>"X.__bases__"</code> will be exactly equal to <code>"(A, B, C)"</code> .                        |
| type.__base__     | <b>**CPython implementation detail:**</b> The single base class in the inheritance chain that is responsible for the memory layout of instances. This attribute corresponds to <code>"tp_base"</code> at the C level. |

```
+-----+-----+
-----+
| type.__doc__          | The class's documentation string, or
"None" if |
|                      | undefined. Not inherited by subclasses.
|
+-----+
```